Python Notes: Type Casting (Type Conversion)

Based on the video: Learn type casting in 7 minutes! 💱

1. What is Type Casting?

MOST IMPORTANT

Type casting is the process of converting a value from one data type to another.

For example, converting an integer (`21`) into a float (`21.0`) or converting a Boolean (`True`) into a string (`"True"`).



Common Python data types

Data type	Description	Example
`str`	String (text)	`"Ashwani"`
`int`	Integer (whole number)	`21`
`float`	Floating-point number	`1.9`
`bool`	Boolean (True or False)	`True`

Why do we need type casting?

Type casting is useful when:

- `Taking user input:` Python's `input()` function returns a string, even when the user enters a number.
- `Performing calculations:` Convert numeric strings into integers or floats before doing arithmetic.
- `Changing data representation:` Convert a number into a string for displaying or combining text.
- `Truncating decimal values:` Convert a float into an integer when you need to remove its fractional part.
- `Checking values:` Convert a value to a Boolean to determine whether it is truthy or falsy.

FREQUENTLY ASKED Why does `input()` + number fail without `int()`? Look at the example below:

Example:

```
age = input("Enter your age: ")
print(age + 1)  # TypeError: str + int is invalid
```

Correct version:

```
age = int(input("Enter your age: "))
print(age + 1)
```

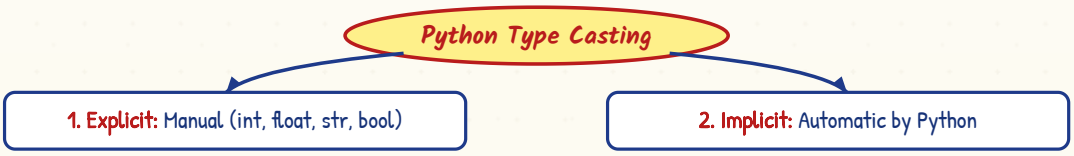
If the user enters `21`, the output is:

```
22
```



2. Types of Type Casting

Python type casting can be divided into two main types.



1. Explicit Type Casting

The programmer manually converts a value using functions such as `int()`, `float()`, `str()` and `bool()`.

```
x = 10
x = float(x)
print(x) # 10.0
```

2. Implicit Type Casting

Python automatically converts a value to a compatible type during certain operations.

```
x = 2
y = 2.0
result = x + y
print(result) # 4.0
```

3. Explicit Type Casting

Explicit type casting means manually converting a value into another data type.

Python provides built-in functions for this purpose.

A. Converting into an integer — `int()`

The `int()` function converts a compatible value into an integer.

Syntax:

```
int(value)
```

Example 1: Float to integer

```
gpa = 1.9
gpa = int(gpa)
print(gpa)
```

Output:

```
1
```

MEMORY TRICK MOST IMPORTANT

Important: `int()` truncates the decimal part; it does not round to the nearest integer.

```
print(int(5.9))  # 5
print(int(5.1))  # 5
print(int(-5.9)) # -5
```

int(5 .9) → 5 (Chops .9) int(-5 .9) → -5 (Towards 0) No rounding happens!

Example 2: String to integer

```
age = "21"
age = int(age)
print(age)       # 21
print(type(age)) # <class 'int'>
```

This works because `"21"` contains a valid integer representation.

However:

```
int("hello") # ValueError
```

B. Converting into a float — `float()`

The `float()` function converts a compatible value into a floating-point number.

Syntax:

```
float(value)
```

Example:

```
age = 21
age = float(age)
print(age)
print(type(age))
```

Output:

```
21.0
<class 'float'>
```

More examples:

```
print(float(10))     # 10.0
print(float("3.14")) # 3.14
print(float(-5))     # -5.0
```

Remember: A float usually displays a decimal point, even when the number has no fractional part.

C. Converting into a string — `str()`

The `str()` function converts a value into its string representation.

Syntax:

```
str(value)
```

Example:

```
student = True
student = str(student)
print(student)
print(type(student))
```

Output:

```
True
<class 'str'>
```

Although the printed output looks like `True`, it is now a string rather than a Boolean.

More examples:

```
print(str(21))     # "21"
print(str(3.14))   # "3.14"
print(str(False))  # "False"
```

Why is it useful?

```
age = 21
print("My age is " + str(age))
```

Output:

```
My age is 21
```

Without `str(age)`, concatenating the string with an integer using `+` would cause a `TypeError`.

D. Converting into a Boolean — `bool()`

The `bool()` function converts a value into either `True` or `False`.

Syntax:

```
bool(value)
```

Rule 1: Numbers

- Zero (`0` , `0.0`) → `False`
- Any non-zero number → `True` , including negative numbers

```
print(bool(10))    # True
print(bool(-5))    # True
print(bool(0))     # False
print(bool(0.0))   # False
```

Rule 2: Strings

- Empty string (`""`) → `False`
- Non-empty string → `True` , including strings containing only spaces

```
print(bool("Ashwani")) # True
print(bool(""))        # False
print(bool(" "))       # True
print(bool("False"))   # True
print(bool("0"))       # True
```

FREQUENTLY ASKED MOST IMPORTANT

Important exam point: `bool("False")` is `True` because `"False"` is a non-empty string. Python does not interpret the text as the Boolean value `False` .

Rule 3: Other common values

```
print(bool(None))  # False
print(bool([]))    # False: empty list
print(bool([1]))   # True: non-empty list
```

FALSY → bool() gives False
0 , 0.0 , "" (empty str) , None , []

TRUTHY → bool() gives True
10 , -5 , " " , "False" , "0" , [1]

Quick reference table

Conversion	Example	Result
Float → integer	int(7.8)	7
Integer → float	float(7)	7.0
Integer → string	str(7)	"7"
String → integer	int("25")	25
String → float	float("2.5")	2.5
Zero → Boolean	bool(0)	False
Non-zero → Boolean	bool(-2)	True
Empty string → Boolean	bool("")	False
Non-empty string → Boolean	bool("Hi")	True

4. The `type()` Function

The built-in `type()` function tells you the data type of a value or variable.

Syntax:

```
type(variable)
```

To display the result, use `print()`.

Example:

```
name = "Ashwani"
age = 21
gpa = 1.9
student = True
print(type(name))
print(type(age))
print(type(gpa))
print(type(student))
```

Output:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Verify a type conversion

```
age = 21
print(type(age))
age = float(age)
print(type(age))
```

Output:

```
<class 'int'>
<class 'float'>
```

MOST IMPORTANT

Key concept: Assigning the result of a conversion back to a variable changes the type of the value currently stored in that variable.

```
age = float(age)
```

You can also convert a value without reassigning it:

```
age = 21
print(float(age)) # 21.0
print(age)        # 21
```

Here, `age` remains an integer because the converted value was not assigned back to it.

5. Implicit Type Casting

Implicit type casting occurs when Python automatically converts a value to another type during an operation.

Example 1: Integer and float arithmetic

```
x = 2
y = 2.0
x = x / y
print(x)
print(type(x))
```

Output:

```
1.0
<class 'float'>
```

Why does this happen?

1. `x` initially stores an integer (`2`).
2. `y` stores a float (`2.0`).
3. Division with `/` produces a float in Python 3.
4. The result `1.0` is assigned back to `x`.
5. Therefore, `x` now refers to a float value.

Example 2: Integer and float addition

```
a = 10
b = 2.5
result = a + b
print(result)       # 12.5
print(type(result)) # <class 'float'>
```

Python converts the integer to a compatible numeric type for the calculation.

Explicit vs. implicit type casting

Feature	Explicit	Implicit
Who performs conversion?	Programmer	Python automatically
How?	Using conversion functions	During supported operations
Example	`float(5)`	`5 + 2.0`
Result	`5.0`	`7.0`

Important: Implicit conversion does not happen for every combination of data types. For example, Python does not automatically convert an integer to a string when concatenating it with text.

```
print("Age: " + 21) # TypeError
```

Instead, convert the integer explicitly:

```
print("Age: " + str(21))
```

6. Common Errors and Important Rules

1. Invalid string conversion

```
int("hello")
```

Raises `ValueError` because `"hello"` is not a valid integer representation.

2. Mixing strings and numbers

```
"Age: " + 21
```

Raises `TypeError`. Use `str(21)` or an f-string instead.

3. Integer conversion does not round

```
int(9.99) # 9
```

Use `round(9.99)` if rounding is what you want.

4. Input is always a string

```
number = input("Enter a number: ")
```

Even if the user types `100`, the result is the string `"100"` until converted.

Bonus: A cleaner way to combine text and numbers

SHORTCUT Use an f-string:

```
age = 21
print(f"My age is {age}")
```

Output:

```
My age is 21
```

An f-string allows you to insert values into text without manually converting each value using `str()`.

7. Practice Questions

Try solving these before checking the answers.

Type Casting Quiz

0/8 ANSWERED

1. What is the output of int(8.9)?

8.9 | 9 | **8** | Error

2. What is the data type of float(5)?

int | str | **float** | bool

3. What is the output of bool("")?

True | **False** | None | Error

4. What is the output of bool("False")?

False | **True** | 0 | Error

5. What is the output of int("25")?

"25" | 25.0 | **25** | Error

6. What happens to the type of x in x = 2; x = x / 2?

It remains int | **It becomes float** | It becomes str | It becomes bool

7. What does type("Hello") return?

int | **<class 'str'>** | <class 'bool'> | Hello

8. Which conversion is explicit?

5 + 2.0 | 10 / 2 | **str(25)** | 2 + 3

CHECK ANSWERS

8. Last-Minute Revision Sheet

Remember these rules

1. **Definition:** Type casting means converting one data type into another.
2. **Explicit:** Manual conversion using `int()`, `float()`, `str()` and `bool()`.
3. **Implicit:** Automatic conversion by Python during supported operations.
4. `int(3.9)` → `3`: The fractional part is truncated.
5. `bool(0)` → `False`, `bool(1)` → `True`.
6. `bool("")` → `False`, `bool(" ")` → `True`.
7. `input()` returns a string. Convert numeric input before performing arithmetic.
8. `type()` identifies a value's data type.

Important questions for exams and interviews

FREQUENTLY ASKED

1. Define type casting with an example.
2. Differentiate between explicit and implicit type casting.
3. Explain the uses of `int()`, `float()`, `str()` and `bool()`.
4. What is the difference between `int(4.8)` and `round(4.8)`?
5. Why does `input()` require type conversion for numerical calculations?
6. Explain the outputs of `bool(0)`, `bool(-1)`, `bool("")` and `bool("False")`.
7. What is the purpose of the `type()` function?
8. Explain implicit conversion using an integer and a float.

MOST IMPORTANT

Final takeaway: Master the four conversion functions — `int()`, `float()`, `str()` and `bool()` — and understand when Python converts numeric types automatically. These concepts are essential for writing correct Python programs, especially when working with user input and calculations.